Indian Sign Language Recognition System using OpenCV and TensorFlow

Garvit Sharma[1], Nayoneeka Paul[2], Runi Ghosh[3], Dr. Dominic Savio M[4], Dr. Pandiyaraju V[5], Dr. Anandan P[6]

[1 2 3] Student, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India

[4] Faculty Supervisor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India

[5 6] Faculty Co-Supervisor, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, India

***Abstract*: An Automatic Sign Language Recognition System is a technology designed to reduce the communication barrier between individuals who rely on sign language and those who do not. The system employs computer vision and machine learning techniques to interpret and translate sign language gestures into text or speech, enabling seamless communication between people with different abilities. In our research, the system achieved a high accuracy rate of 90% for the classes 'A', 'G', 'E', 'B', and 'C', demonstrating its proficiency in recognizing these gestures. For the classes 'D', 'F', 'H', 'M', 'L', 'J', '0', 'P', 'X', and 'Z', the system maintained an accuracy rate between 80% and 90%, showing reasonable effectiveness. However, certain classes fell within an accuracy range of 80% to 85%, indicating specific areas where further optimization is needed. The system achieved F1 scores of approximately 0.946, 0.916, and 0.896 for classes with 90%, 85%, and 82% accuracy, respectively, highlighting its balanced performance across different gesture categories. Designed to be user-friendly and accessible, the system offers real-time translation capabilities across various devices and platforms, such as smartphones and wearables. This sign language recognition system contributes to Human-Computer Interaction (HCI) by providing a practical and inclusive solution, ultimately aiming to enhance accessibility and inclusivity, bridging the communication gap between hearing and hard-of-hearing individuals.**

Keywords- Sign Language Recognition, OpenCV, CNN, SIFT, DWT, TensorFlow, Keras.

1 Introduction

The mute does not speak while the deaf do not hear. Despite these limitations they are capable of performing and excelling at various tasks. The key feature that sets them apart is the means for communication.

Deaf and mute people use sign language to express themselves. People are often ignorant about this technique unless it concerns their own family or friends. Sign language interpreters aid the deaf and mute but are exorbitant. These people require an affordable technique to communicate to the other section of society with no hindrance.

As a result, researchers play an important role in assisting the deaf and mute population to converse fluently with the rest of the society . The breakthrough in the research conducted was the SLR System [17]. The SLR System uses text or speech to translate sign language to a local language. This system is expensive and difficult to operate and manage. Data gloves [8] which record the movement of the fingers and hand while making sign language gestures using some sensors has aided early research into Sign Language Recognition Systems. Because of the gloves' high cost and wearability, commercialization is difficult. Further development by the researchers were pure vision, but the hand tracking remained hard.

Picture acquisition and classification issues plague sign language recognition. They are constantly being researched and worked upon. Image pre-processing is difficult with cameras and active sensors are costly. For the investigators, this classification has drawbacks. Due to the variety, researchers are unable to select a single recognition method. Concentrating on a single strategy prevents testing other methods that may be more effective for Sign Language Recognition. Researchers develop multiple strategies while experimenting with various pre-existing datasets.

This article will discuss the SLR System developed by the researchers. Our study will go over the devices used to collect data, data acquisition, and discuss the recent recognition methods used by researchers [16].

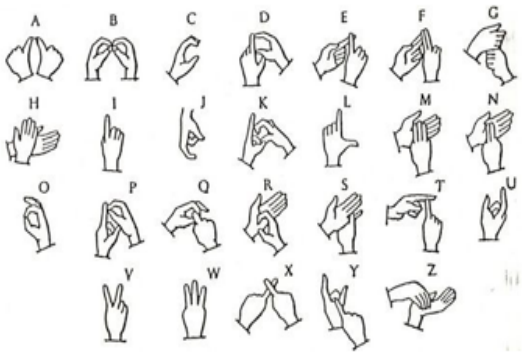


Figure 1: Indian Sign Language

Figure 2: American Sign Language

1.1 Sign Language Symbols

The symbols mentioned in Figure 1 and Figure 2 [15] represent the Indian and American Sign Language Symbols respectively and can indicate all the aspects of a sign, including hand forms, movement, position, and palm orientation. In sign language, signs can be classified as one-handed or two-handed based on the number of hands used to make the sign.

One-handed signs are made using only one hand as mentioned in Figure 2. These signs are typically made with the dominant hand, and the non-dominant hand is often used to support the sign or to indicate additional information such as location or movement. Examples of one-handed signs include the signs for "dog," "cat," "book," and "please."

Two-handed signs as mentioned in Figure 2 are made using both hands. These signs often involve movements or positions that are difficult to make with one hand alone. Examples of two-handed signs include the signs for "write," "read," "love," and "I."

Static hand gestures in sign language refer to those that are held in one position, whereas dynamic hand gestures refer to those that involve movement or change in position. Examples of static hand gestures in American Sign Language (ASL) include the signs for letters such as "A" or "S", while an example of a dynamic hand gesture in ASL is the sign for "drinking" which involves the movement of the hand from an open position to a closed position at the mouth.

In Two-handed signs, both the dominant and non-dominant signs are employed. This allows two handed signs to be represented. These are further broken down into type 0 sign and type 1 sign categories. In the type 0 sign, both hands are actively producing movements, whereas in the type 1 sign, the dominant hand produces more movements than the non-dominant hand.

It's worth noting that sign language varies between countries, cultures and even regions, so the signs mentioned here might not be the same in other sign languages.

Figure 3: Subdivisions of Indian Sign Language

2 Classification of ISL Signs

Figure 3 shows how the ISL constituents are detailed classified. The signs come into two broad categories depending on whether one or two hands are used. Within each category, the signs are further classified according to their movement properties: static signs do not exhibit any motion, and dynamic signs are defined by their movement. In this respect, the diagram examines the relationship of manual to non-manual signs in ISL. Handshapes whose use involves only one hand can be categorized under either as manual or non-manual. Meaning is encoded in manuals through handshapes, orientations, locations, and movements. Non-manual signs use facial expressions and body language to add or alter meaning in a sign. There are two major categories for two-handed signs: Type 0 and Type 1. In the former, there is little difference in what both hands do. In Type 1, the action or handshape with one hand is not the same as with the other, usually specifying some kind of relationship or interaction between the two concepts. In summary, Figure 3 represents the intricate composition of ISL, highlighting the interplay between manual gestures, facial expressions, and body language in the formation of a nuanced and expressive linguistic system. This conceptual model offers a significant basis for comprehending the complexities of ISL and its varied linguistic features.

3 Methodology

An expanded methodology for a sign language recognition system could include Image Acquisition, Preprocessing, and Data Augmentation.

3.1 Image Acquisition:

Image Acquisition is a process that, as its name suggests, comprises the acquisition of images during runtime through the utilization of an inbuilt webcam. Once the photographs have been acquired, they are immediately placed in a directory for storage. Using the SIFT (Scale Invariant Feature Transform) algorithm [4][12], the recently acquired image is then compared to photos that have been previously captured and saved for a certain letter in a database. This comparison reveals both the gesture that was really performed as well as the text that was translated to correspond with that move. The photographs are taken with the assistance of fundamental programming in MATLAB [15], which opens the user's webcam and takes a picture of the scene at a predetermined rate of frames per second. All of the input photographs, including these newly captured ones, are maintained in a directory that is separate from the rest of the images. After then, the most recent image that was captured is chosen and compared to a collection of photographs. The user interface of the application includes a button labeled "START," which, when pressed, turns the built-in webcam on. After activation, the button reverts to its previous state, "STOP." After the user has finished the gesture, the frame will be taken and saved in the directory once it has been processed.

3.2 Preprocessing:

Preprocessing in a sign language recognition system is the process of preparing the data collected from the sign language gestures for use in the system's model. Some common preprocessing steps that may be performed in a sign language recognition system include:

3.2.1 Image processing:

Once an image has been captured, it will typically undergo processing to remove noise, improve the image's overall quality, and extract features that are pertinent to the process of recognition. This may involve cropping the image so that the hand motion is the primary emphasis, altering the brightness and contrast, and removing any background components that could potentially obstruct or confuse the recognition process.

3.2.2 Feature extraction:

Figure 4 represents a sequence of frames depicting a person signing are provided. Images (a) RGB-in the original color image, (b) depth map-making known which points on the person's body are at what distance from the camera, and (c) user index-indicating which position on the screen the user is-. Skeleton of the person: It is a skeleton representation of the person highlighting the major joints and areas where they are located. This is used as a basis for evaluation of handshapes, orientations, and movements which are beneficial in sign language analysis. These extracted features are used further for processing and classification in the sign language recognition system.

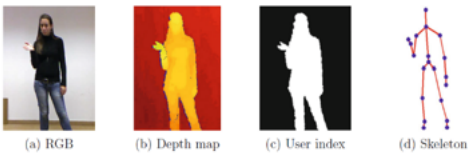


Figure 4: Feature Extraction

3.2.3 Data Normalization:

The system may normalize the data to guarantee that the input data is in a consistent manner that the model is able to comprehend and use efficiently. This could involve scaling or normalizing the data so that it conforms to a standard size or format, or it could involve deleting any outliers that could potentially impede the recognition process.

3.2.4 Data splitting:

To train and evaluate the model, the system may divide the data into training and testing sets. All these steps can be done using libraries like OpenCV, NumPy, and scikit-learn. The act of Preprocessing is essential for the system to function effectively and accurately, as it guarantees that the data is formatted in a way that the model can utilize optimally.

3.3 Data Augmentation:

The term "data augmentation" refers to a method that is utilized in the process of artificially expanding the size of a training dataset as we can see in Figure 5. This is particularly useful in sign language recognition systems, as obtaining a large and diverse dataset of sign language gestures can be challenging. By applying various transformations to the images in the dataset, such as flipping, rotating, and scaling, the model can be exposed to a wider variety of data during training, which can help improve its ability to generalize to new, unseen images of sign language gestures.

Image flipping as shown in Figure 5 is one specific technique that can be used for data augmentation in a sign language recognition system. By flipping the images in the dataset horizontally or vertically, the model can be exposed to additional examples of the signs, allowing it to learn the variations of the signs from different perspectives.

Rotation is another technique that can be used for data augmentation. By rotating the images in the dataset to different angles, the model can be exposed to examples of signs made at different orientations, allowing it to learn how to recognize the signs regardless of the angle at which they are presented.

Scaling is another technique that can be used for data augmentation. By scaling the images in the dataset up or down, the model can be exposed to examples of signs made at different sizes, allowing it to learn how to recognize the signs regardless of their size.

Adding noise is another technique that can be used for data augmentation. By adding random noise to the images as in Figure 5 in the dataset, the model can be exposed to examples of signs made under different lighting conditions or with different types of camera noise, allowing it to learn how to recognize the signs under different conditions.

Cropping is another technique that can be used for data augmentation as we can see in Figure 5. By randomly cropping the images in the dataset, the model can be exposed to examples of signs made with different background or hand gesture positions, allowing it to learn how to recognize the signs regardless of the background or hand gesture positions.

Therefore, by using data augmentation techniques, the model will be exposed to a wider variety of data during training, which can help improve its ability to generalize to new, unseen images of sign language gestures. This can help prevent over fitting, which occurs when a model becomes too specialized to the training data and performs poorly on new, unseen data.

Figure 5: Examples of data augmentation

4 Related Works

[1] Sunitha et al. (2016) examines various techniques used in communication interpreter systems for deaf-mute individuals, divided into two categories: online learning systems and wearable communication devices. Wearable devices, including those based on gloves, keypads, and touchscreen technology, utilize sensors, accelerometers, micro-controllers, and speech synthesis. These devices are practical and provide tangible feedback, but they can be cumbersome and less comfortable for continuous use. On the other hand, online learning systems like SLIM, TESSA, Wi-See, SWI_PELE, and Web-Sign aim to eliminate the need for external devices, offering a more seamless user experience. However, these systems may lack the immediacy and precision that wearable devices can provide.

[2] According to Anand et al. (2016) the ISLR method was proposed for recognizing patterns in sign language. This technique consists of two primary elements: feature extraction and data classification. By utilizing the DWT for feature extraction in combination with the nearest neighbor classifier, the ISLR approach can effectively interpret sign language, achieving a high accuracy rate of 99.23% using the cosine distance classifier. The strength of this method lies in its precision, but its reliance on DWT makes it computationally intensive, which could be a limitation in real-time applications.

[3] Ahuja and Singh (2015) presented a database-centric approach to hand gesture recognition, using an effective template matching mechanism based on a skin color model and thresholding approach. This method, which can be applied in both human robotics and similar applications, employs the skin color model in the YCbCr color space to segment the hand area, followed by thresholding to differentiate between the foreground and background. Finally, a template-based matching algorithm that utilizes PCA for recognition is applied. The strength of this approach is its effectiveness in controlled environments, but it may struggle with varying lighting conditions and diverse skin tones, which can affect accuracy.

[4] More and Sattar (2015) introduced a hand gesture recognition system founded on digital image processing, intended for static hand movements. They utilized the SIFT algorithm to examine the feature vector of hand gestures, calculating SIFT features at the edges, which are robust against scaling, rotation, and the introduction of noise. The advantage of this method is its robustness to various image transformations, making it reliable under different conditions. However, the SIFT algorithm's computational complexity can be a drawback, particularly in applications requiring real-time processing.

[5] IJSR (2018) describes a technique for automatically identifying signs using shape-based attributes, developed by the authors. Otsu's thresholding technique is used to extract the hand region from images, aiming to minimize the within-class variation of thresholded black and white pixels by selecting an appropriate threshold. The segmented hand region's features are then extracted using Hu's invariant moments, which are input into an ANN for classification. The system's performance is measured using three metrics: accuracy, sensitivity, and specificity. The strength of this approach is its ability to accurately classify signs based on shape, but its sensitivity to variations in hand shape and position may limit its effectiveness across different users.

[6] Kaur and Gill (2019) discussed numerous methods for identifying hand gestures and sign languages that have previously been proposed by various scholars. Sign language is the primary mode of communication for those who are deaf, mute, or both, allowing these individuals to express their emotions and thoughts to others. While these methods are effective in facilitating communication, they often require specialized equipment or controlled environments, which can limit their accessibility and practicality in everyday situations.

[7] Pandey and Jain (2015) used Indian Sign Language (ISL) to develop a technique that helps deaf and mute individuals communicate with other ISL users by translating relevant text messages from hand gestures. The main goal is to create an algorithm that can instantly translate dynamic motion into text, which will be achieved through the development of this algorithm. Once all testing is complete, the system will be modified for use on the Android platform, leading to the creation of a mobile application for smartphones and tablets. The strength of this approach lies in its potential for real-time translation and mobile accessibility, but it requires extensive testing and optimization to handle the complexity of dynamic gestures effectively.

[8] Wadhawan et al. (2021) employed a strategy in this study which considered factors such as acquisition mode, static/dynamic signs, signing mode, single-handed/double-handed signs, techniques used, and average accuracy. A camera was used as the mode of acquisition, covering both single-handed and double-handed sign usage, although the study primarily focused on static signs. The method utilized in this study is CNN, with the assistance of feature extraction techniques HOG and SIFT, yielding a recognition rate of 92.33%. The strength of this approach is its high recognition rate, but the reliance on CNNs requires large datasets and significant computational resources, which could be a limitation in less resource-rich environments.

[9] In the first step of this paper, Pigou et al. (2014) extracted low-level features like edges and corners, followed by the extraction of higher-level functionality by the upper layer. The input has dimensions of N*N*D, and it is processed with H kernels, each with the same dimensions as the input: k*k*D. When one input is convolutionally processed with one kernel, one output feature is produced; however, when H kernels are independently processed, H output features are generated. The acquisition mode used in the paper is a camera, covering both static and double-handed static signs. The technique used here is CNN. The strength of this method lies in its layered feature extraction process, which enhances recognition accuracy, but the computational demands of CNNs may pose a challenge for real-time applications or on devices with limited processing power.

[10] Halder et al. (2021) focuses on replacing the old SLR technique with a more comfortable and less restrictive system. The author demonstrates that this system functions without any sensors, making it extremely cost-effective compared to alternative techniques. The focus is on using MediaPipe and a machine learning algorithm to create a pipeline that is quicker, easier, more affordable, portable, and simple to implement. MediaPipe is used to preprocess photos, providing multi-hand landmarks, while the Panda library is utilized to clean and normalize the data. SVM is then used to make the predictions, with the system achieving a 99% accuracy rate. The strength of this method is its cost-effectiveness and portability, but it may struggle with more complex sign languages or in environments with significant variations in lighting or background.

[11] Anand et al. (2016) presented an automated ISLR system based on DWT. In the process of computing sub-band energy features, DWT is utilized up to the 7th level to ensure that the resulting feature set is as valuable as possible. Using the cosine

distance measure, this technique reaches an accuracy of 99.23%. The strength of this system is its high accuracy, but its reliance on DWT may limit scalability and real-time application, particularly in less powerful processing environments.

[12] Sharma et al. (2013) employs SIFT for feature matching and uses a built-in webcam for data collection. The paper includes the calculation of keypoints, followed by comparing them with the database, generating the output for the provided sign based on the comparison. The strength of this method is its effective feature matching, but SIFT's processing demands could hinder its applicability in real-time systems, where speed is crucial.

[13] Rahman et al. (2019) utilizes a collection of 2425 images for ASL, which are then preprocessed by converting them to grayscale and resizing them to 64 by 64 pixels. The CNN model has a total of 7 layers, including 6 convolutional layers, 3 pooling levels, and 1 fully connected layer. For data augmentation, the images are rotated between -10° and 10°, zoomed by 10%, and shifted by 10% on both height and breadth. Each dataset is randomly divided into K=10 folds, with K-1 used for training and the remaining fold for measuring the model's performance. The system achieves a precision of 100%. The strength of this approach is its extremely high precision, but it relies heavily on extensive preprocessing and a large, varied dataset, which could be a limitation in more constrained settings.

[14] Bheda et al. (2017) proposes using Microsoft's Kinect device for depth sensing to address issues related to skin tone. The architecture of the CNN includes two groups of fully connected layers, followed by a dropout layer and an output layer. Additionally, three sets of two convolutional layers are followed by a max-pool layer and then a dropout layer. The dataset includes 25 photographs contributed by 5 individuals, covering each letter and digit of the alphabet. The system achieves an accuracy of 82.5% for recognizing letter motions and 97% for recognizing digits. The strength of this method is its ability to handle depth sensing, but its effectiveness may vary based on the user's hand positioning and the surrounding lighting conditions.

[15] Patel and Ambekar (2017) This work describes moment-based sign language gesture recognition, with a comprehensive literature survey reviewing sign language recognition systems, including vision-based, depth-sensor, and other methods. Sign language recognition is particularly important in India, where various dialects and languages can make communication between hearing-impaired and hearing individuals difficult. The proposed method extracts features from the segmented hand region using Hu's seven moment invariants, with several studies employing moment-based features for SLR, including PCA and SVMs. The proposed method was used to recognize sign language motions in Hindi, Gujarati, and Marathi, achieving high recognition rates. However, the method may face challenges due to the diverse dialects and gestures across Indian languages, which can affect its overall effectiveness.

[16] Chong et al. (2018) describes a real-time ISL gesture recognition system. The literature review covers vision-based, data glove-based, and accelerometer-based sign language recognition algorithms. The proposed system employs a camera to collect hand movements and recognize ISL, achieving 95.63% accuracy on 480 ISL gestures. The strength of this system is its high accuracy, but it is sensitive to lighting conditions and requires the user to perform gestures within a specific visual region, which limits its flexibility in more dynamic environments. [18].

[17] Suharjito et al. (2017) presents a comprehensive review of computer-based techniques for recognizing Indian Sign Language (ISL). The survey includes methods like hidden Markov models, artificial neural networks, and support vector machines. Each method has its strengths and weaknesses. Hidden Markov models are effective in handling temporal variations in gestures but require large datasets for training, which can be a limitation in practice. Artificial neural networks are powerful in capturing complex patterns, but they are computationally intensive and require significant processing power, which might not be available in all applications. Support vector machines, on the other hand, offer good generalization capabilities and are less prone to overfitting, but they may struggle with large-scale problems where the number of gestures or signs is extensive. The study concludes that while these methods have been successful in ISL recognition, ongoing challenges such as varying lighting conditions, different skin tones, and the need for large, labeled datasets remain. Raghuveera et al. [19].

[20] Sivabalaselvamani et al. (2024) focuses on developing a machine learning model to detect and translate Indian Sign Language (ISL) into spoken language, aiming to facilitate communication between deaf and dumb individuals and those who do not understand sign language. The proposed system uses a camera to capture sign language, which is then processed using TensorFlow's Object Detection API to predict and translate the signs. The strength of this approach lies in its user-friendliness and cost-effectiveness, as it only requires a camera and does not depend on additional hardware like gloves or sensors. Additionally, the system is designed to operate in real-time, providing immediate translation, which is a significant improvement over existing solutions that are either not real-time or only moderately accurate. However, a key limitation is that the model's performance may be constrained by the size and diversity of the training dataset, potentially affecting its accuracy in recognizing a wide range of ISL motions and hand poses. Despite this, the approach shows promise in reducing communication barriers for the disabled community.

[21] Indu et al. (2024) focuses on creating an advanced American Sign Language (ASL) recognition system to improve communication for the deaf and hard-of-hearing community. By using the 'grassknoted/asl-alphabet' Kaggle dataset and training a Convolutional Neural Network (CNN), it aims to achieve a 99% accuracy rate in recognizing 29 different hand gestures, including alphabet signs and specific actions like 'delete' and 'space.' The project, spanning ten weeks, involves data collection, model development, testing, and interface integration. While the approach promises high accuracy and real-time performance, its effectiveness may be limited by the quality and diversity of the training dataset.

5 Proposed Work

The Proposed Work consists of four major steps which are Image Acquisition, Hand Distinguishing, Skeleton Detection, and Gesture Detection which are also mentioned in Figure 6.

In the later portion of the piece, each following phase is carefully discussed, offering specific information on the operation of the module as well as the sort of behavior it is reportedly meant to display.

We have gathered 16,000 photos for the dataset that will be utilized in the system. There are a total of 36 classes, and each class contains the letters A through Z and the numerals 0 through 9.

We've used libraries like OpenCV, CVZone, and NumPy in order to successfully develop the ISL recognition model and Teachable Machines (TensorFlow Keras) to train the model.

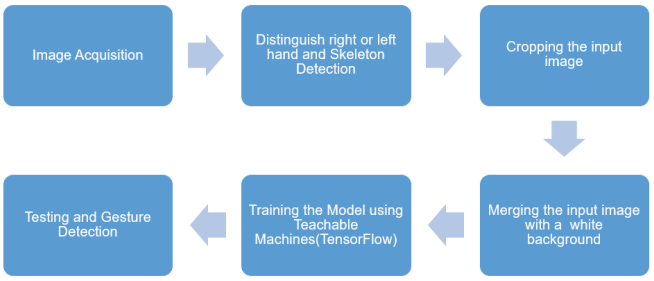


Figure 6: Flow chart of Proposed System

5.1 Image Acquisition

The first step of the proposed is Image Acquisition. As the name suggests, this is the process of acquiring the images for our System. Some researchers believe that gesture gloves are superior to web-cameras for capturing gesture images, but this comes at the expense of increased complexity in using the gloves and higher costs associated with their purchase and maintenance of the gesture gloves. So, in order to minimize the complexity of using data gloves and reducing the cost of maintenance and acquisition, we are acquiring images for our work using the integrated camera during runtime.

5.2 Distinguishing between Left and Right Hand, and Skeleton Detection

For a system to correctly discern the sign that a person is attempting to make with their hands, it is important for the system to have the ability to precisely distinguish and discriminate between a person's right hand and left hand. We are utilizing the Hand Detector function that is a part of the CVZone's Hand Tracking Module so that we can do this assignment with the best possible degree of precision. This is a Python module that is frequently utilized in software applications that are concerned with Computer Vision and Image Processing.

Following this, we will search for the skeleton of the hand to discriminate between the left hand and the right hand in a manner that is both effective and accurate. This is essential because, as seen in Figure 7, the skeletons of both hands are unique and mirror images of one another, making them incompatible with one another.

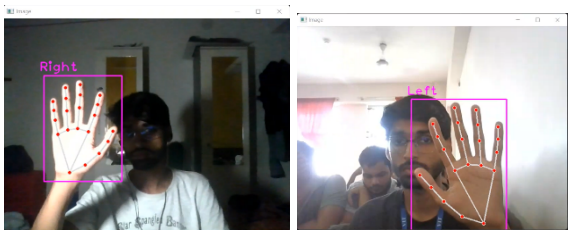


Figure 7: Identifying if a hand is right or left one.

The successful identification of the hand will hand in better classification and will lead to higher rate of accuracy which will make the model more usable in real world.

5.3 Cropping the image (Pre-processing)

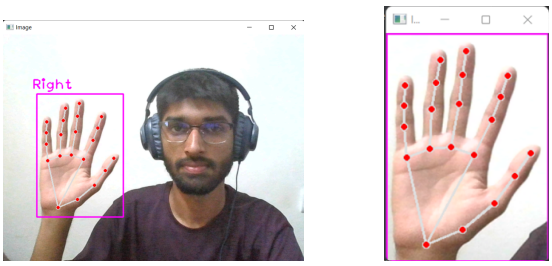
In a variety of applications that are linked to hand gesture detection and analysis, one crucial step in the pre-processing phase is cropping a picture to extract the hand gesture signals that were produced by the signer.

The following are a few reasons that illustrate the significance of cropping as an integral aspect of the pre-processing stage:

- *Focus on Relevant Information*: By cropping the image such that just the hand gesture area is visible, you may get rid of the distracting backdrop features and concentrate entirely on the movements themselves. This gets rid of distractions and noise, making it possible for future algorithms to concentrate on the essential aspects of the hand motions.
- *Reduced Complexity*: The complexity of the next steps in the processing is reduced by cropping. The algorithms can focus their attention on a smaller, more particular section of the picture rather than the complete image itself. This decreases the amount of computing that is required and speeds up the analytical process.
- *Improved Accuracy*: The cropping process results in a more accurate and clear representation of the hand motions. In order to improve the signal-to-noise ratio and make it simpler for identification and classification algorithms to correctly comprehend hand gestures, extraneous background clutter should be removed.
- *Consistent Hand Gesture Representation*: Cropping offers a consistent portrayal of hand motions across several datasets and samples. You may remove variances brought about by varied backdrops or unnecessary items by focusing on the important hand region. This makes it much simpler to compare and analyze hand motions in a consistent manner.
- *Minimized Interference*: The act of cropping a photograph can assist remove distractions caused by other aspects of the subject's body or items that are visible in it. By isolating the region of the hand motion, you limit the risk of incorrect categorization brought on by occlusions or nearby objects.
- *Efficient Resource Utilization*: Cropping cuts down on the amount of computer resources that are needed for future processing. Processing durations are reduced, resulting to increased real-time performance in applications such as gesture recognition systems. This is made possible by a smaller cropped zone, which allows the succeeding algorithms to operate more quickly.

Since the primary focus of our study is on the recognition of static sign language, the only part of the input image that would be useful to us would be the area containing the hands. Therefore, we need to crop the image to extract the required region before moving on to the next step in the process.

The cropping of the desired hand region from an input image is demonstrated in Figure 8.



(A) (B)

Figure 8: (A) The input image from the camera. (B) Cropped the desired area out of the input image.

As shown in Figure 8, we are extracting the region of interest which is hand sign in our case from the input image bounding box. Then, the region inside the bounding box is cropped and extracted from the input image. This is done to achieve simplicity as the model will not have to look for signs anywhere else other than the bounding box image.

This will also help for better classification in the later stages of the paper.

5. 4 Merging the cropped image with a White Background

Since the proportions of the various signs vary, which means that some may be longer or wider than others, it is essential that we maintain consistency across the board regarding the signs' appearance. To put it into practice, we have employed an approach that begins with a white backdrop that is 300X300 pixels in size. After that, we use the picture that has been cropped after it has been taken from the input image. After that, we combine both photos in order to guarantee that all of the images that are extracted from the input image adhere to consistent dimensions and do not differ from one another in terms of their length or breadth.

Merging the cropped image containing the hand gesture signs with a white background of 300 pixels can be beneficial to ensure uniformity for several reasons:

- *Standardization*: You may create a canvas with standardized size for the hand gesture indications by using a white backdrop with predetermined dimensions (for example, 300 pixels). This assists in the creation of consistent visual representations across a variety of pictures or samples, which makes it much simpler to compare and analyze the hand motions.
- *Aspect Ratio*: The use of a white backdrop with predetermined proportions helps to keep the hand gesture indications inside a constant aspect ratio. This is of utmost significance when contrasting and interpreting motions that entail changes in the form of the hand or the location of the fingers. Because of the set dimensions, the proportions of the hand motions will always be the same, despite the fact that the size of the original picture or its aspect ratio may change.
- *Simplified Analysis*: Working with photos of the same size throughout the analysis and processing processes streamlines the overall process. It makes it possible for algorithms to anticipate a consistent input format, which makes the integration of new components and the standardization of existing ones simpler. Hand gesture recognition and categorization are two examples of activities that might benefit from this improvement in efficiency and accuracy.
- *Background Removal*: Putting an image on top of a white backdrop makes it much easier to get rid of the background of the original picture. It helps differentiate the hand gesture indications from the components or items that are present in the original image that are in the background. This has the potential to improve the segmentation process, making it much simpler to zero in on the hand movements themselves as the primary emphasis.
- *Visual Consistency*: A white backdrop that is consistent throughout creates a portrayal of the hand gesture indications that is aesthetically clean and free of clutter. It ensures that the attention is kept completely on the hand movements by removing any potential distractions or variances that may have been brought about by the various backdrops that were used in the original photographs.

Figure 9 shows a visual representation of the process.

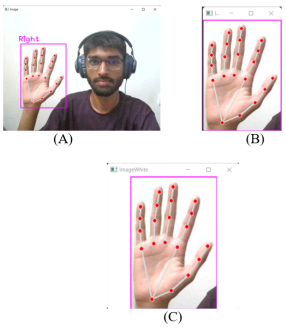


Figure 9: Extracting the image from the bounding box and merging with a White background.

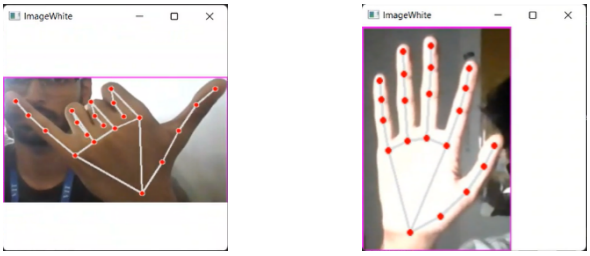


Figure 10: Images with different dimensions merged with white background.

Figure 10 demonstrates that even though the images differ in their dimensions, merging them on top of a white background image that is 300x300 pixels will ensure that the images are uniform. Although one image is greater in height and the other one is greater in width, after being merged with a white background image, both images are converted to 300x300 pixels.

5.5 Training the Model using teachable machines

In order to train the model, we've used Teachable Machine which is a web-based tool developed by Google.

Teachable Machine enables users to train machine learning models without coding. It supports tasks such as image and sound classification and pose estimation. Users can gather labeled examples, train the model using transfer learning, and utilize the model in real-time applications. It serves as an accessible platform for hands-on learning and experimentation with machine learning concepts.

The model was trained with all the 36 classes and was downloaded for testing purposes as shown in figure 11.

The dataset is to be divided into classes so that all the images belonging to a particular class are uploaded together.

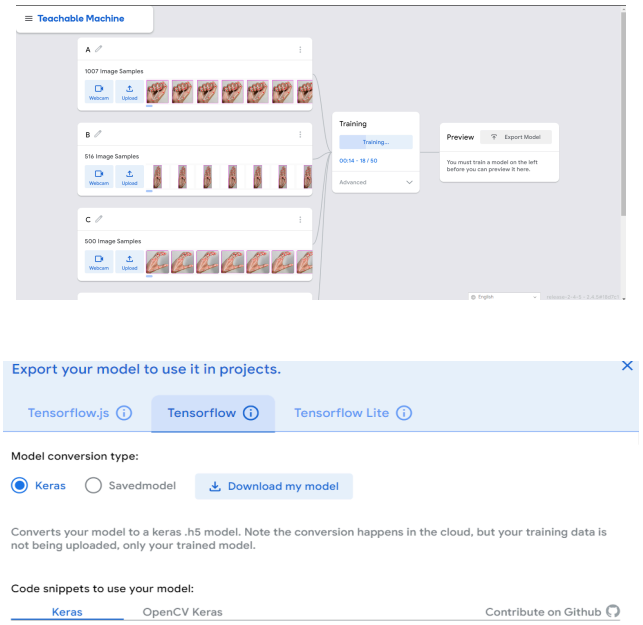


Figure 11: Training and saving the model.

5.6 Testing and Gesture Detection

When we evaluated the trained model, we were able to get an accuracy of around 84.24% on average when we tested it. The model did a good job of predicting the courses, and it was able to achieve varying levels of accuracy for each group of classes. More specifically, the model displayed a high accuracy rate of around 90% for the classes 'A', 'B', 'C', 'E', 'G'. This suggests that the model did a good job of properly projecting the performance of these classes.

In addition, the model displayed an accuracy rate of around 85% for the classes 'D', 'F', 'H', 'J', 'L', 'M', 'O', 'P', 'X' respectively. This accuracy rate nevertheless suggests that a relatively trustworthy forecast can be made for these classes, even though it is somewhat lower than the rate for the first group.

However, the model experienced considerable difficulties while attempting to classify the letters 'K,' 'N,' 'Q,' 'R,' 'S,' 'T,' 'U,' 'V,' 'Y,' as seen by the significantly lower average accuracy rate it attained, which was roughly 82%. Because of the reduced accuracy rate for these classes, the overall average accuracy of the model was around 84.24%

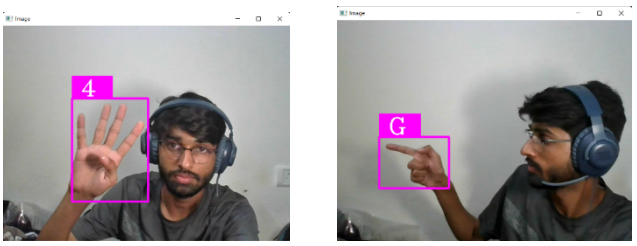


Figure 12: Predicting the letter 'G' and number '4'.

Figure 12 shows the correct prediction of the model as the figure contains the model predicting the letter 'G' and the number '4' correctly without much difficulty.

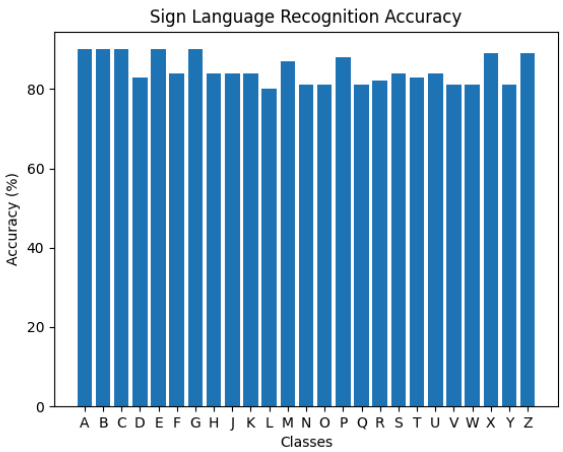


Figure 13: Accuracy rate of each class.

Figure 13 is a representation of the accuracy rate of each class, which indicates how accurately and successfully the model can predict that specific class.

The overall accuracy of the model can be increased if the classes that are giving out lesser accurate predictions will be worked upon in the future and while increasing the training data in the future works.

F1-Score Extraction: A statistic called the F1 score is used to assess how well a classification model is performing, particularly in situations when there is an imbalance in the distribution of classes. It offers a solitary score that harmonises memory and precision.

Key Concepts

1. **Precision**:
   - Measures how many of the predicted positive instances are actually positive.
   - Formula:

$$Precision = \frac{TP}{TP+FP}$$

   -
   - Where TP is True Positives and FP is False Positives.

2. **Recall**:
   - Measures how many of the actual positive instances are correctly predicted.
   - Formula:

$$Recall = \frac{TP}{TP+FN}$$

   - Where TP is True Positives and FN is False Negatives.

F1-Score Formula: The F1 score combines precision and recall into a single metric. It is the harmonic mean of precision and recall, providing a balance between the two:

$$F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Why Use F1 Score?

Balanced Metric: It takes false positives and false negatives into account, which is helpful when one kind of error is more important than another.

Handling Imbalanced Data: Especially useful in cases of imbalanced datasets whereby one class is far more frequent than others.

Calculation of F1

I have around 1000 images per class, let me calculate the FP and f1- score and other necessary parameters for performance evaluation.

Total images per class: 1000

Accuracy for classes 'A', 'B', 'C', 'E', 'G': 90%

Accuracy for classes 'D', 'F', 'H', 'J', 'L', 'M', 'O', 'P', 'X': 85%

Accuracy for classes 'K', 'N', 'Q', 'R', 'S', 'T', 'U', 'V', 'Y': 82%

FP:

Classes with 90% Accuracy

Total images per class: 1000, Accuracy: 90%, True Positives (TP): 900, False Negatives (FN): 100

To find FP:

$$FP = \frac{Total\ incorrect\ predictions}{Number\ of\ classes} = \frac{100}{36} \approx 2.7$$

Classes with 85% Accuracy

Total images per class: 1000, Accuracy: 85%, True Positives (TP): 850, False Negatives (FN): 150

$$FP = \frac{Total\ incorrect\ predictions}{Number\ of\ classes} = \frac{150}{36} \approx 4.1$$

Classes with 82% Accuracy

Total images per class: 1000, Accuracy: 82%, True Positives (TP): 820, False Negatives (FN): 180

$$FP = \frac{Total\ incorrect\ predictions}{Number\ of\ classes} = \frac{180}{36} = 5$$

To calculate the F1 score for each group of classes based on the given information, we use the following formulas:

$$Precision = \frac{TP}{TP+FP}$$

$$Recall = \frac{TP}{TP+FN}$$

$$F1\ score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1-Score:

Classes with 90% Accuracy

True Positives (TP): 900, False Negatives (FN): 100, False Positives (FP): 2.7

$$Precision = \frac{900}{900+2.7} \approx 0.997 \; Recall = \frac{900}{900+100} = 0.9$$

$$F1 = 2 \times \frac{0.997 \times 0.90}{0.997+0.90} \approx 0.946$$

Classes with 85% Accuracy

True Positives (TP): 850, False Negatives (FN): 150, False Positives (FP): 4.1

$$Precision = \frac{850}{850+4.1} \approx 0.995 \; Recall = \frac{850}{850+150} = 0.85$$

$$F1 = 2 \times \frac{0.995 \times 0.85}{0.995+0.85} \approx 0.916$$

Classes with 82% Accuracy

True Positives (TP): 820, False Negatives (FN): 180, False Positives (FP): 5

$$Precision = \frac{820}{820+5} \approx 0.991$$

$$Recall = \frac{820}{820+180} = 0.82$$

$$F1 = 2 \times \frac{0.991 \times 0.82}{0.991+0.82} \approx 0.896$$

Summary of F1 Scores

- **Classes with 90% Accuracy**: F1 Score ≈ 0.946
- **Classes with 85% Accuracy**: F1 Score ≈ 0.916
- **Classes with 82% Accuracy**: F1 Score ≈ 0.896

6 Conclusion and future works

6.1 Summary of Findings:

The performance analysis of the sign language recognition (SLR) model reveals promising results with room for further enhancement. The model demonstrated high accuracy for the classes 'A', 'B', 'C', 'E', and 'G', achieving an accuracy rate of 90%. This indicates strong proficiency in predicting these specific signs. For classes 'D', 'F', 'H', 'J', 'L', 'M', 'O', 'P', 'X', and 'Z', the model achieved an accuracy between 80% and 90%, reflecting a solid performance across a broader range of signs.

However, the model's performance for classes with accuracy rates below 85% suggests that there are specific areas requiring improvement. The calculated F1 scores reinforce this observation:

- **Classes with 90% Accuracy:** F1 Score ≈ 0.946
- **Classes with 85% Accuracy:** F1 Score ≈ 0.916
- **Classes with 82% Accuracy:** F1 Score ≈ 0.896

These scores highlight that while the model performs excellently for certain classes, there is a noticeable decline in performance for others, particularly for those with lower accuracy.

6.2 Future Works:

To enhance the model's performance and address its current limitations, the following strategies are recommended:

6.2.1 Class-Specific Model Optimization:

- ***Objective*:** Tailor the model for each class to better handle its unique characteristics.
- ***Approach*:** Adjust hyperparameters, implement class-specific data augmentation techniques, and refine training strategies to target the specific challenges of each class. This might involve creating separate models for difficult classes or employing specialized training techniques.

6.2.2 Data Augmentation:

- ***Objective**:* Improve the model's ability to generalize across different variations of sign language gestures.
- ***Approach***: Expand the dataset using techniques such as rotation, scaling, and translation to simulate various hand shapes, orientations, and lighting conditions. This will help the model become more robust to real-world variations in sign language gestures.

6.2.3 Fine-Tuning with Transfer Learning:

- ***Objective:*** Leverage existing knowledge from related tasks to improve model performance.
- ***Approach:*** Use pre-trained models from image classification or hand gesture recognition tasks as a foundation. Fine-tune these models on the sign language dataset to benefit from learned features and improve accuracy.

6.2.4 Increasing Training Data*:*

- ***Objective**:* Enhance model generalization by providing a more diverse set of training examples.
- ***Approach:*** Collect additional data that represents a broader range of individuals, hand sizes, skin tones, and backgrounds. This will help the model perform better across diverse scenarios and improve its robustness.

6.2.5 Model Ensembling*:*

- ***Objective:*** Improve overall accuracy and robustness by combining predictions from multiple models.
- ***Approach:*** Implement ensemble methods such as majority voting or stacking to integrate predictions from various models trained on different subsets of the data. This approach can help balance out the weaknesses of individual models and enhance overall performance.

When these enhancements are implemented, we may anticipate improved accuracy and resilience in ISLR, which will result in these systems being more dependable and effective in real-world settings and that would make the work even more applicable and useful, while also making it easier for a greater number of individuals who are in need to get their hands on it.

7 References

[1] Sunitha K. A, Anitha Saraswathi.P, Aarthi.M, Jayapriya. K, Lingam Sunny, "Deaf Mute Communication Interpreter- A Review", International Journal of Applied Engineering Research ,Volume 11, pp 290-296 , 2016.

[2] Mathavan Suresh Anand, Nagarajan Mohan Kumar, Angappan Kumaresan, "An Efficient Framework for Indian SignLanguage Recognition Using Wavelet Transform" Circuits and Systems, Volume 7, pp 18741883, 2016.

[3] Mandeep Kaur Ahuja, Amardeep Singh, "Hand Gesture Recognition Using PCA", International Journal of Computer Science Engineering and Technology (IJCSET), Volume 5, Issue 7, pp. 267-27, July 2015.

[4] Sagar P.More, Prof. Abdul Sattar, "Hand gesture recognition system for dumb people", 2015.

[5] International Journal of Science and Research (IJSR), 2018.

[6] Chandandeep Kaur, Nivit Gill, "An Automated System for Indian Sign Language Recognition", International Journal of Advanced Research in Computer Science and Software Engineering, 2019.

[7] Pratibha Pandey, Vinay Jain, "Hand Gesture Recognition for Sign Language Recognition: A Review", International Journal of Science, Engineering and Technology Research (IJSETR), Volume 4, Issue 3, March 2015.

[8] Wadhawan, Ankita, and Parteek Kumar. "Sign language recognition systems: A decade systematic literature review." Archives of Computational Methods in Engineering 28.3 (2021): 785-813.

[9] Pigou L, Dieleman S, Kindermans PJ, Schrauwen B. Sign language recognition using convolutional neural networks. In European conference on computer vision 2014 Sep 6 (pp. 572-578). Springer, Cham.

[10] Halder, Arpita, and Akshit Tayade. "Real-time vernacular sign language recognition using mediapipe and machine learning." Journal homepage: www. ijrpr. com ISSN 2582 (2021): 7421.

[11] Anand, Mathavan Suresh, Nagarajan Mohan Kumar, and Angappan Kumaresan. "An efficient framework for Indian sign language recognition using wavelet transform." Circuits and Systems 7.8 (2016): 1874-188

[12] Sharma, Shanu & Goyal, Sakshi & sharma, ishita. (2013). Sign Language Recognition System For Deaf And Dumb People. International Journal of Engineering Research and Technology. 2. 382-387.

[13] M. M. Rahman, M. S. Islam, M. H. Rahman, R. Sassi, M. W. Rivolta and M. Aktaruzzaman, "A New Benchmark on American Sign Language Recognition using Convolutional Neural Network," 2019 International Conference on Sustainable Technologies for Industry 4.0 (STI), 2019, pp. 1-6, doi: 10.1109/STI47673.2019.9067974

[14] Bheda, Vivek & Radpour, Dianna. (2017). Using Deep Convolutional Networks for Gesture Recognition in American Sign Language.

[15] U. Patel and A. G. Ambekar, "Moment Based Sign Language Recognition for Indian Languages," *2017 International Conference on Computing, Communication, Control and Automation (ICCUBEA)*, Pune, India, 2017, pp. 1-6, doi: 10.1109/ICCUBEA.2017.8463901.

[16] Chong, Teak-Wei & Lee, Boon Giin. (2018). American Sign Language Recognition Using Leap Motion Controller with Machine Learning Approach. Sensors. 18. 3554. 10.3390/s18103554.

[17] Suharjito, Ricky Anderson, Fanny Wiryana, Meita Chandra Ariesta, Gede Putra Kusuma. "Sign Language Recognition Application Systems for Deaf-Mute People: A Review: Based on Input-Process-Output", Procedia Computer Science, 2017

[18] H. Muthu Mariappan and V. Gomathi, "Real-Time Recognition of Indian Sign Language," 2019 International Conference on Computational Intelligence in Data Science (ICCIDS), Chennai, India, 2019, pp. 1-6, doi: 10.1109/ICCIDS.2019.8862125.

[19] Raghuveera, T., Deepthi, R., Mangalashri, R. and Akshaya, R., 2020. A depth-based Indian sign language recognition using microsoft kinect. *Sādhanā*, *45*, pp.1-13.

[20] Sivabalaselvamani, D., D. Selvakarthi, L. Rahunathan, S. Gokulprasath, D. Jagane, and S. Logeshwar. "Implementation of Converting Indian Sign Language into Indian Language Using IoT-based Machine Learning Algorithms." Advances in Computational Intelligence and Its Applications (2024): 99.

[21] Indu, M., K. Surya Sathwika, T. Raeshma Sree, KVD N. Sai Kiran, and S. Syam Kumar. "American Sign Language Recognition System.", 2024.